

Combining Workload Aware Partitioning with Streaming Graph Partitioning

Vladimir M. Kozlov

© The Author(s)

Abstract Efficient graph partitioning is a critical factor in achieving high performance and scalability in distributed graph databases. Conventional graph partitioning methods, such as streaming graph partitioning and METIS, rely exclusively on the structural properties of the graph and do not account for graph access patterns or query workloads. As a result, these methods primarily optimize partition quality by minimizing the number of inter-storage edges (cut size). However, edges crossing storage boundaries may be frequently accessed during query execution, leading to increased communication overhead and degraded query performance. This paper presents a comparative analysis of workload-aware graph partitioning methods and their integration with streaming graph partitioning. The evaluated approaches are compared in terms of partition quality and their impact on query execution performance in distributed graph databases.

Keywords graph partitioning, distributed graph databases, streaming graph partitioning, Fennel, Kernighan–Lin algorithm, SCHISM, WAWPart, graph algorithms, workload aware partitioning

1 Introduction

The increasing volume and complexity of graph-structured data have led to the widespread adoption of distributed graph databases in domains such as social networks, recommendation systems, knowledge graphs, and cybersecurity. By distributing graph data across multiple storage nodes, these systems can scale to billions of vertices and edges while supporting parallel query execution. However, the efficiency of a distributed graph database largely depends on the quality of graph partitioning. Poor partitioning increases the amount of inter-node communication during query execution, leading to higher latency and reduced overall system performance.

Traditional graph partitioning algorithms, including METIS [?], Kernighan–Lin, and more recent streaming approaches such as Fennel [?], primarily optimize graph

structure by minimizing the number of edges crossing partition boundaries while maintaining balanced partition sizes. Although minimizing the cut size is an important objective, it does not necessarily correspond to improved query performance. In many real-world applications, only a subset of graph edges is frequently traversed. Consequently, even a partitioning with a relatively small cut size may result in substantial communication overhead if frequently accessed edges are distributed across different partitions. An example of this situation is shown in Fig. 1, where the colored edges represent the most frequently traversed edges. The partitioning on the left minimizes the edge cut; however, many of the cut edges are heavily traversed during query execution, resulting in a large number of Inter-Partition Traversals (IPTs). In contrast, the partitioning on the right has a bigger edge cut but significantly fewer IPTs.

To address this limitation, several workload-aware partitioning techniques have been proposed. These methods incorporate information about the actual query workload into the partitioning process, seeking to reduce the number of inter-partition traversals during query execution. Examples include Schism [?] and WAWPart [?], which optimize data placement according to expected access patterns rather than relying solely on graph topology.

This paper investigates whether combining structural and workload-aware partitioning techniques can improve partition quality from the perspective of query execution. Specifically, we evaluate combinations of the Fennel streaming partitioning algorithm with the Kernighan–Lin refinement algorithm, Schism, and WAWPart. The evaluated approaches are compared using the Inter-Partition Traversal (IPT) metric, which measures the number of graph traversals crossing partition boundaries during query execution. Unlike traditional graph cut metrics, IPT directly reflects the communication overhead experienced by distributed graph databases. Furthermore, unlike execution time, IPT is independent of the implementation details and performance characteristics of a particular graph database system, making it a more objective metric for evaluating partitioning quality.

This paper presents a comparative evaluation of hybrid

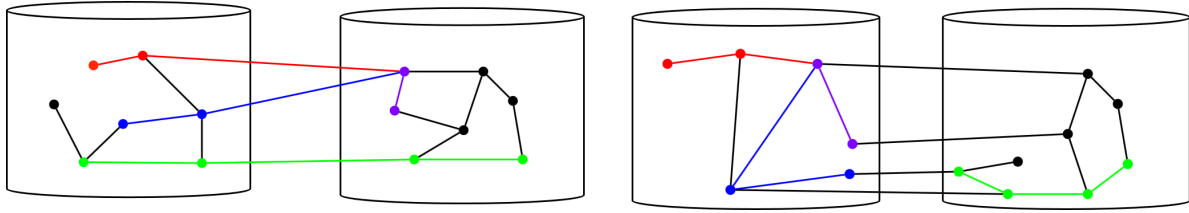


Fig. 1 Example illustrating that a minimum edge cut does not necessarily minimize inter-partition traversals. Colored edges represent the most frequently traversed edges.

graph partitioning strategies that combine the Fennel streaming partitioning algorithm with refinement and workload-aware optimization techniques, including Kernighan–Lin, Schism, and WAWPart.

2 Combined methods review

One of the methods reviewed in this paper is Schism [?]. Without considering implementation details, the core idea of Schism is to assign a special weight to each edge, referred to as the request weight (r-weight). The r-weight of an edge is increased every time the corresponding edge is traversed during query execution. As a result, frequently accessed edges gradually obtain higher weights. Instead of minimizing the number of inter-storage edges or the size of the edge cut, Schism aims to minimize the total weight of the cut. Therefore, edges with high r-weights, representing frequently accessed relationships, are more likely to be placed within the same storage node, while rarely accessed edges are allowed to remain between partitions.

Another workload-aware partitioning method considered in this paper is WAWPart [?]. Although the original work does not explicitly discuss Schism and does not provide a direct comparison with it, WAWPart is based on a similar concept: assigning r-weights to edges and increasing these weights according to query usage. The main difference is that WAWPart does not increase the weights of all traversed edges equally. Instead, the increase depends on the traversal order, where edges encountered earlier receive larger weight increments, while later edges receive smaller increments. The partitioning objective remains the same: minimizing the total weight of the cut.

The similarity between Schism and WAWPart raises the question of their relative effectiveness and whether workload-aware optimization provides significant advantages compared with traditional minimum-cut partitioning approaches.

To utilize the optimization principles of the previously described workload-aware methods, an algorithm capable of minimizing weighted cuts is required. A suitable candidate is METIS [?], a state-of-the-art graph partitioning framework for minimizing edge cuts.

METIS extensively utilizes the Kernighan–Lin (KL) refinement algorithm, which evaluates the benefit of moving a vertex to another partition using the following heuristic:

$$g_v = \sum_{(v,u) \in E \wedge P[v] \neq P[u]} w(v,u) - \sum_{(v,u) \in E \wedge P[v] = P[u]} w(v,u).$$

This heuristic represents the difference between the total weight of edges connecting a vertex to external partitions and the total weight of edges connecting it to its current partition. In other words, KL inherently minimizes the total weight of the cut and therefore can be applied as a refinement stage after workload information has been collected.

However, METIS has relatively high computational overhead due to its multilevel coarsening, initial partitioning, and uncoarsening phases. Therefore, in this work, these phases are replaced with the streaming graph partitioning algorithm Fennel. Due to the nature of streaming partitioning, Fennel alone cannot perform workload-aware partitioning because vertices and edges are assigned during graph loading, before their future usage patterns are known. Workload-aware behavior can only be introduced if additional semantic information about vertices is available or if the graph is reloaded together with workload-derived information. Such approaches are outside the scope of this paper.

Also our experiments has shown Fennel indeed provide good partitioning that can be used for refinement by KL without requiring large amount of iterations of it.

Therefore, the complete partitioning process is divided into three phases:

- (1) Initial graph loading, where vertices are distributed using Fennel or random partitioning;
- (2) Workload application, where queries are executed and edge usage information is collected;
- (3) Iterative refinement using Schism, WAWPart, or no refinement.

3 Experiments data generation

The experiments were conducted on synthetically generated random geometric undirected graphs. For each graph size

(5 000, 10 000, 20 000, 50 000, 100 000, and 250 000 vertices), the average vertex degree was fixed at 7, resulting in graphs of similar density while varying only the number of vertices. Each graph was partitioned into different numbers of storage nodes (4, 6, 8, 10, 12 and 16).

Queries were generated as just pairs of vertices of the graph between which is needed to find a path. To imitate real query patterns as close as possible the pairs were chosen in the following patterns:

- (1) absolutely random pairs (40%);
- (2) random pair that geometrically not further than determined distance between each other (30%);
- (3) pair in which one of vertices is considered a center of clusters (20%);
- (4) pair within one cluster; (5%);
- (5) random pair in determined number of edges from each other (5%).

4 Experiments

The primary evaluation metric was the graph cut percentage, defined as the percentage of edges whose endpoints belong to different partitions. A smaller graph cut corresponds to less inter-partition communication during graph traversal.

Each graph was processed incrementally. Vertices were inserted sequentially using the Fennel partitioning algorithm. After every 1 000 inserted vertices, the current partitioning was refined using the Kernighan–Lin (KL) algorithm. This interval was selected as a compromise between partition quality and computational overhead. The graph cut was measured after the entire graph had been processed. For comparison, the same graphs were also partitioned using Fennel without any refinement.

Figure 2 shows that KL refinement consistently reduces the graph cut for all evaluated graph sizes. Although the relative improvement decreases for larger graphs, the reduction in graph cut remains significant.

An additional observation concerns the convergence of the KL algorithm. For all evaluated graph sizes, the algorithm typically converged after only two refinement iterations for each pair of partitions. Only a small number of cases required three iterations, and no experiment required more than three. This supports the observation reported in the original METIS work that a good initial partitioning substantially reduces the amount of refinement required [?].

The experiments were performed only on synthetic graphs of modest size. Evaluating the proposed approach on real-world graph datasets and larger graphs remains part of future work.

These observations indicate that Fennel produces an initial partitioning that is sufficiently close to a locally optimal solution, allowing KL refinement to converge rapidly while still reducing the graph cut.

5 Conclusion

This work investigated a hybrid graph partitioning approach that combines the scalability of the Fennel streaming partitioning algorithm with the local optimization capabilities of the Kernighan–Lin refinement algorithm. The proposed method replaces the computationally expensive coarsening and uncoarsening stages of the METIS framework with online partitioning using Fennel, while periodically applying KL refinement to improve partition quality.

The experimental evaluation demonstrated that the proposed approach consistently reduced the graph cut compared to the partitioning produced by Fennel alone. Furthermore, the refinement process typically converged after only two iterations for each pair of partitions and never required more than three iterations in the evaluated experiments. These observations support the hypothesis that the partitioning generated by Fennel is of sufficiently high quality for efficient KL refinement, allowing the computational cost of refinement to remain low while improving partition quality.

Although the current evaluation was performed on synthetic graphs of moderate size, the results indicate that the proposed hybrid approach represents a practical compromise between partition quality and computational efficiency. It is therefore well suited for distributed graph databases and other systems that require scalable online graph partitioning while minimizing inter-partition communication.

Acknowledgments

// TODO:

Declaration of Competing Interest

// TODO:

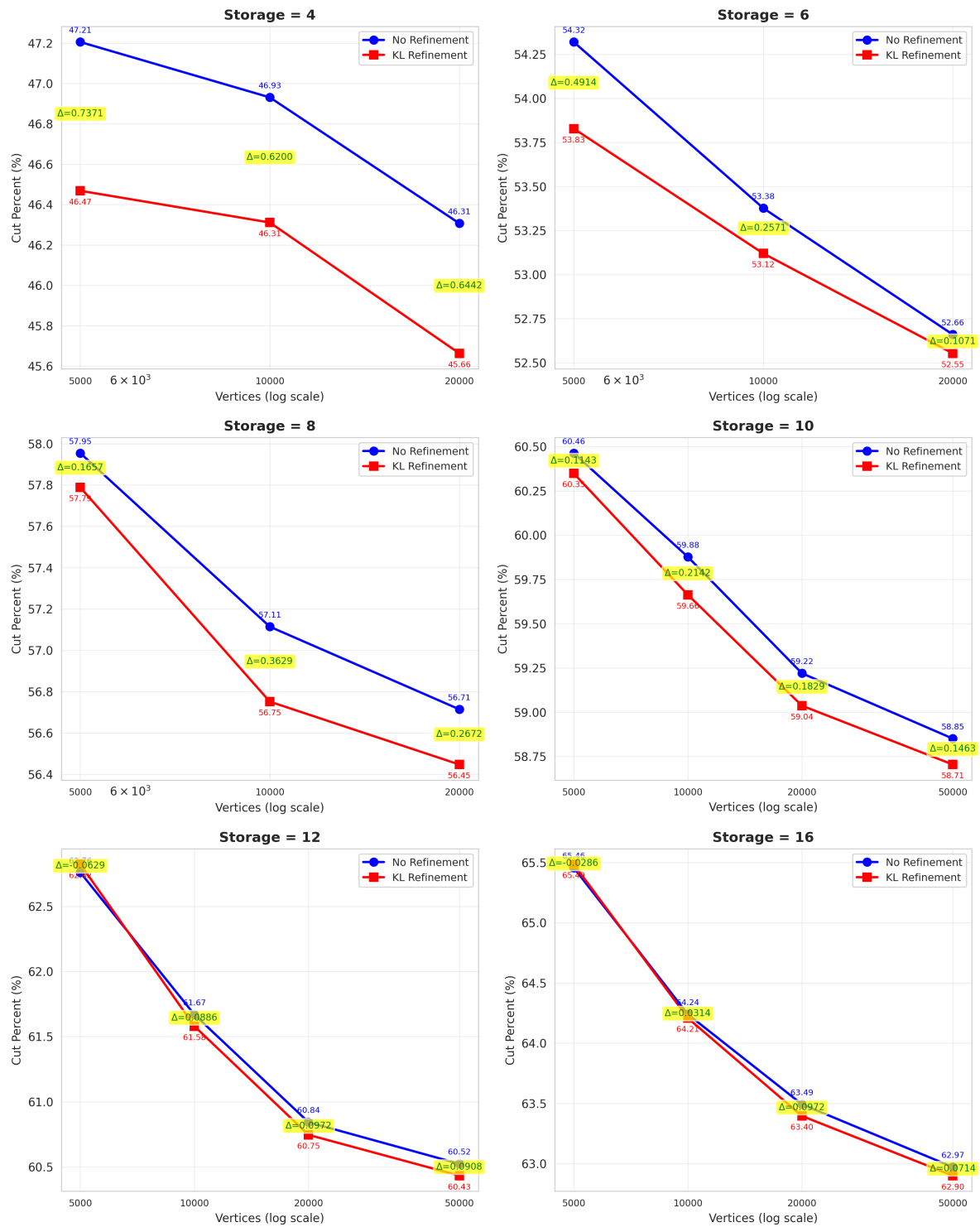


Fig. 2 Results of the experiments.